

Implémentation de structure dynamique hiérarchique en C: Les Arbres

On désire écrire un programme qui permet de manipuler la structure dynamique hiérarchique Arbre binaire de recherche.

A l'exécution, le programme doit présenter le menu suivant

- Créer un arbre binaire de recherche. (les nœuds contiennent des entiers saisis au clavier)
- Affichage du résultat du parcours en profondeur de l'arbre (en infixe)
- Déterminer la profondeur de l'arbre
- Ajouter un nœud à l'arbre binaire de recherche
- Supprimer un nœud
- Rechercher un nœud

Travail à Faire

- 1- Donnez en C l'implémentation du type qui permet de manipuler un arbre binaire d'entiers

CORRECTION

```
struct Noeud{
    struct Noeud* FG;
    struct Noeud* FD;
    unsigned int info;
};
typedef struct Noeud Noeud;
typedef Noeud* ArbreB;
```

- 2- Ecrire la fonction main avec affichage du menu et appels des différents modules

```
#include <stdio.h>
#include <stdlib.h>
void AjoutNoeudABR ( ArbreB *R, ArbreB NN ) ;
void ParcoursInfixe ( ArbreB R);
void remplirArbre ( ArbreB *R );
void supprim ( ArbreB *R, int x);
int profondeur(ArbreB R);
int Recherche ( ArbreB R, int x);
int MinABR(ArbreB R) ;
int MaxABR(ArbreB R);
void main ( ) {
    int rep=1, maxi, mini, taille, prof,x, trouve;
    ArbreB R=NULL, NN, ee;
    do {
        printf("\n 1 =====> Remplir Arbre Binaire de recherche");
        printf("\n 2 =====> Afficher parcours en Infixe");
        printf("\n 3 =====> Recherche un élément ");
        printf("\n 4 =====> Supprimer un élément");
        printf("\n 5 =====> Insérer un élément");
        printf("\n 6 =====> Afficher la Profondeur ");
        printf("\n 0 =====> QUITTER ");
        scanf("%d", &rep);
        switch (rep) {
            case 1: R = NULL;
                remplirArbre(&R);
                break;
            case 2: if (R)
                ParcoursInfixe(R);
                else
                printf("\n L'arbre est VIDE");
                break;
```

```

case 3: printf("\n Donnez l'élément à chercher dans l'arbre : ");
        scanf("%d", &x);
        trouve = Recherche(R, x);
        if (trouve)
            printf("\n %d Appartien à l'arbre ", x);
        else
            printf("\n %d N'appartient pas à l'arbre ", x);
        break;
case 4: printf("\n Donnez l'élément à supprimer de l'arbre : ");
        scanf("%d", &x);
        supprim(&R, x);
        break;
case 5: NN = malloc(sizeof(Noeud));
        printf("\n Donnez l'élément à Insérer dans l'arbre : ");
        scanf("%d", &NN->info);
        NN->FG=NULL;
        NN->FD = NULL;
        AjoutNoeudABR(&R, NN);
        break;
case 6: prof = Profondeur (R);
        printf("\n Profondeur Arbre = %d ", prof);
        break;
    }
} while (rep);
}

```

- 3- On vous rappelle ci-dessous les différents algorithmes de manipulations d'un arbre binaire de recherche

Procédure AjoutNoeudABR (var R : ArbreB; NN : ArbreB)

Début

```

    Si (R = nil) alors
        R ← NN
    Sinon
        Si (*NN .info ≤ *R.info) alors
            AjoutNoeudABR(*R.FG, NN)
        Sinon
            AjoutNoeudABR(*R.FD, NN)
        FinSi
    finSi
FinSi

```

Fin AjoutNoeudABR

```

void AjoutNoeudABR ( ArbreB *R, ArbreB NN ) {
    ArbreB fils=*R;
    if ((*R)==NULL)
        *R=NN;
    else {
        if ( NN->info <= fils->info)
            AjoutNoeudABR( &((*R)->FG), NN);
        else
            AjoutNoeudABR( &((*R)->FD), NN);
    }
}

```

Fonction RechercheABR(R : ArbreB; x : entier) : booléen

Var

trouve : booléen

Début

trouve $\leftarrow$ FauxSi (R $\neq$ nil) alors

Si (*R.info = x) alors

trouve $\leftarrow$ vrai

Sinon

Si (x < *R.info) alors

trouve $\leftarrow$ RechercheABR(*R.FG, x)

Sinon

trouve $\leftarrow$ RechercheABR(*R.FD, x)

FinSi

FinSi

FinSi

Retourne (trouve)

Fin RechercheABR

```

int Recherche ( ArbreB R, int x) {
    int trouve = 0;
    if ( R) {
        if (R->info == x)
            trouve =1;
    }
    else {
        if ( x < R->info)
            trouve = Recherche(R->FG, x);
        else
            trouve = Recherche(R->FD, x);
    }
    }
    return (trouve);
}

```

Fonction Profondeur (R : ArbreB) : Entier

Var

p, pg, pd : entier

Début

p $\leftarrow$ 0Si (R $\neq$ nil) alorsSi (*R.FG $\neq$ nil) alorsp $\leftarrow$ 1 + Profondeur (*R.FG)

Finsi

Si (*R.FD $\neq$ nil) alorsp1 $\leftarrow$ 1 + Profondeur (*R.FD)

si (p1 > p) alors

p $\leftarrow$ p1

FinSi

FinSi

FinSi

Retourne (p)

Fin Profondeur

```

int Profondeur(ArbreB R) {
    int pg=0, pd = 0;
    if (R) {
        if (R->FG)
            pg= 1 + Profondeur (R->FG);
        if (R->FD)
            pd = 1 + Profondeur (R->FD);
        if ( pg > pd )
            pd = pg;
    }
    return (pd);
}

```

Procédure SuppNoeudABR(var R : ArbreB; x : entier)

Var

Max, Min : entier

Début

Si (R ≠ nil) alors

Si (*R.info = x) alors

Si (*R.FG = nil) et (*R.FD=nil) alors

Libérer (R)

R ← nil

Sinon

Si (*R.FG ≠ nil) alors

Max ← MAXABR(*R.FG)

*R.info ← Max

SuppNoeudABR(*R.FG, Max)

Sinon

Min ← MINABR(*R.FD)

*R.info ← Min

SuppNoeudABR(*R.FD, Min)

FinSi

FinSi

Sinon // x non encore trouvé

Si (*R.info < x) alors

SuppNoeudABR(*R.FD, x)

Sinon

SuppNoeudABR(*R.FG, x)

FinSi

FinSi

FinSi

Fin SuppNoeudABR

CORRECTION

```

int MinABR(ArbreB R) {
    if (!R->FG)
        return (R->info);
    else
        return MinABR(R->FG);
}

```

```

int MaxABR(ArbreB R) {
    if (!R->FD)
        return (R->info);
    else
        return MaxABR(R->FD);
}

void supprim ( ArbreB *R, int x) {
    int Max, Min;
    if (*R) {
        if ((*R)->info == x) {
            if (!(*R)->FG) && (!(*R)->FD) {
                free (*R); *R = NULL;
            } else {
                if (*R)->FG {
                    Max = MaxABR((*R)->FG);
                    (*R)->info = Max;
                    supprim( &(*R)->FG, Max);
                } else {
                    Min = MinABR((*R)->FD);
                    (*R)->info = Min;
                    supprim( &(*R)->FD, Min);
                }
            }
        }
        else { // x non encore trouvé
            if ((*R)->info < x)
                supprim(&(*R)->FD, x);
            else
                supprim(&(*R)->FG, x);
        }
    } else printf("\n L'arbre est VIDE");
}

void remplirArbre(ArbreB *R) {
    ArbreB NN=NULL;
    int x, i, n;
    printf("\n Donnez le nombre de noeuds : ");
    scanf("%d", &n);
    for(i=0;i<n;i++) {
        NN=malloc(sizeof(struct Noeud));
        NN->FG=NULL;
        NN->FD=NULL;
        printf("\n Donnez noeud : "); scanf("%u", &(NN->info));
        AjoutNoeudABR(R, NN);
    }
    printf(" \n Fin saisie ");
}

void ParcoursInfixe ( ArbreB R) {
    if (R !=NULL) {
        ParcoursInfixe(R->FG);
        printf("\n Racine = %u", R->info);
        ParcoursInfixe(R->FD);
    }
}

```